

RELIABLE MQTT FOR BEGINNERS

เริ่มใช้ reliomq จากศูนย์

คู่มือสำหรับคนที่ยังไม่เคยใช้ MQTT หรือเขียน Python มาก่อน: ต้องเปิด
อะไรบ้าง ข้อมูลวิ่งไปไหน และรู้ได้อย่างไรว่าส่งถึงแล้ว

สำหรับ reliomq 0.6.2

ตัวอย่างทดลองบนเครื่องเดียวด้วย Mosquitto

เริ่มจากภาพใหญ่ก่อน

เป้าหมายของเราคือส่งค่าจาก Sensor ไปยังโปรแกรมปลายทาง โดยถ้าเครือข่ายหลุดชั่วคราว ข้อความยังมีโอกาสถูกเก็บและส่งซ้ำได้

ในการทดลองนี้ต้องมี 4 งานกำลังทำงาน:

- 1) MQTT broker รับ-ส่งข้อความ
- 2) Relay พาข้อมูลไปปลายทาง
- 3) Subscriber แทนโปรแกรมปลายทาง
- 4) Sensor Sender สร้างและส่งข้อมูล

คำศัพท์แบบไม่ใช่ภาษายาก

MQTT broker

เหมือนศูนย์ไปรษณีย์ โปรแกรมส่งข้อความมาฝากตามชื่อช่องที่เรียกว่า topic แล้ว broker แจกต่อให้ผู้ที่ติดตามช่องนั้น

Sender

ผู้ส่งของฝั่ง Sensor รับข้อมูลจากโค้ดเรา สร้างหมายเลขข้อความ เก็บข้อความที่ยังไม่จบงาน และลองส่งซ้ำเมื่อจำเป็น

Relay

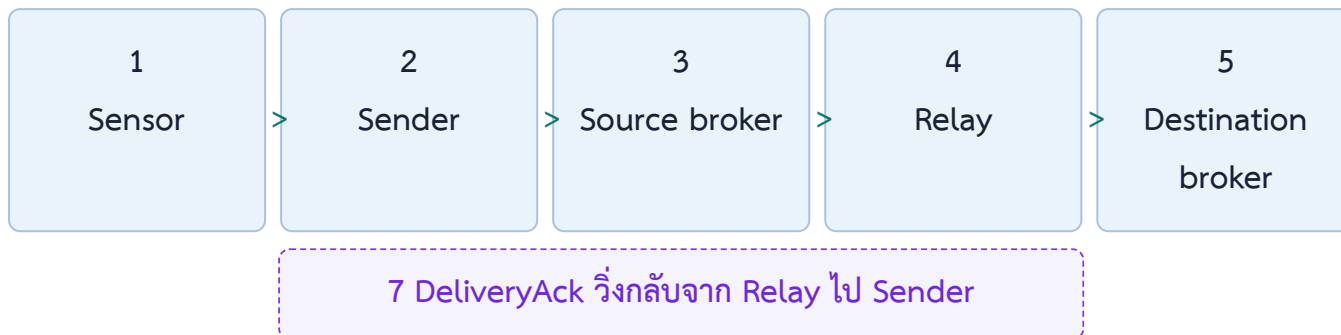
คนกลางที่รับของจาก Sender เปิดดูว่าปลายทางคือ topic ไหน แล้วส่งข้อมูลจริงต่อไปยัง destination broker

Subscriber

โปรแกรมที่รอรับข้อมูลจาก application topic ในระบบจริงอาจเป็นฐานข้อมูล dashboard หรือระบบแจ้งเตือน

ทำไมต้องมี Relay? Sender ต้องการคำยืนยันแบบ DeliveryAck ว่าข้อมูลผ่านไปถึง broker ปลายทางแล้ว Relay เป็นผู้ส่งคำยืนยันนี้กลับมา การมี broker อย่างเดียวจึงยังไม่ครบ workflow ของ reliomq

ข้อมูลเดินทางอย่างไร



1. Sensor สร้างข้อมูล เช่น `{"temperature": 27.4}`
2. โค้ดเรียก `sender.publish("factory/demo/sensor01", data)` ชื่อ topic ในวงเล็บคือ ปลายทางจริง
3. Sender ใส่ข้อมูล, application topic และ `message_id` ลงใน envelope แล้วส่ง envelope ไป `relay_topic` ที่ source broker
4. Relay ติดตาม `relay_topic` จึงได้รับ envelope และอ่านว่าต้องส่งข้อมูลไป application topic ไດ
5. Relay publish ข้อมูลจริงไป `factory/demo/sensor01` ที่ destination broker
6. destination broker ส่ง MQTT PUBACK ให้ Relay ว่ารับ publish แล้ว
7. Relay ส่ง **DeliveryAck** กลับมาที่ `delivery_ack_topic` เมื่อ Sender พบ ACK ที่มี message ID ตรงกัน จึงถือว่างานของข้อความนั้นจบ

จุดที่คนเริ่มต้นมักสับสน:

`relay_topic` และ `delivery_ack_topic` เป็นช่องภายในของ reliomq ส่วน `factory/demo/sensor01` คือช่องข้อมูลจริงที่โปรแกรมปลายทางต้อง subscribe

ทดลองเครื่องเดียวกับระบบจริงต่างกันอย่างไร

	คู่มือนี้	ระบบจริง
Source broker	<code>localhost:1883</code>	อาจเป็น IP ของโรงงาน/อุปกรณ์
Destination broker	broker ตัวเดิมบน localhost	อาจเป็น cloud หรือ server อีกเครื่อง
ปลายทาง	<code>mosquitto_sub</code> เพื่อให้เราเห็นข้อมูล	database, dashboard หรือ service ของคุณ

แม้ทดลองด้วย broker ตัวเดียว แต่ให้คิดว่า source และ destination เป็นคนละบทบาท Relay ยังคงรับแล้วส่งต่อเหมือนระบบจริง

ก่อนเขียนโค้ด: เตรียมของให้ครบ

1. โปรแกรมที่ต้องมี

- Python 3.11 ขึ้นไป ใช้รัน Sender และ Relay
- Mosquitto broker ใช้เป็นศูนย์รับ-ส่ง MQTT
- mosquitto_sub ใช้ดูว่าข้อมูลถึง application topic แล้ว
- Terminal คือหน้าต่างสำหรับพิมพ์คำสั่ง เช่น PowerShell หรือ Terminal บน Raspberry Pi

2. ตรวจสอบว่าโปรแกรมพร้อม

```
python --version
mosquitto -h
mosquitto_sub -h
```

ถ้าคำสั่งใดขึ้นว่าไม่พบโปรแกรม ให้ติดตั้งโปรแกรมนั้นก่อน บน Raspberry Pi/Debian ใช้:

```
sudo apt update
sudo apt install -y mosquitto mosquitto-clients
sudo systemctl enable --now mosquitto
```

3. สร้างโฟลเดอร์ทดลองและติดตั้ง reliomq

```
mkdir reliomq-demo
cd reliomq-demo
python -m venv venv

# Windows PowerShell
venv\Scripts\Activate.ps1

# Linux / Raspberry Pi ใช้บรรทัดนี้แทน
source venv/bin/activate

pip install reliomq==0.6.2
```

หลัง activate สำเร็จ มักเห็นคำว่า **(venv)** หน้าบรรทัดคำสั่ง ทุก terminal ที่ใช้รัน Python ในคู่มือนี้ต้อง activate environment เดียวกันก่อน

ต้องเปิดกี่หน้าต่าง

เปิด terminal แยกกันเพื่อให้แต่ละงานทำต่อเนื่องพร้อมกัน อย่าปิด terminal ของ Relay หรือ broker ระหว่างทดสอบ

Terminal 1

Broker

`mosquitto -v`

ถ้า Mosquitto ทำงานเป็น service อยู่แล้ว ไม่ต้องเปิด terminal นี้

Terminal 2

Relay

`python relay.py`

ต้องเปิดค้างไว้เพื่อรับ envelope และส่ง DeliveryAck

Terminal 3

ตัวดูข้อมูลปลายทาง

`mosquitto_sub -h localhost -t "factory/demo/sensor01" -v`

Terminal 4

Sensor Sender

`python sensor_demo.py`

สร้างค่าจำลองแล้วส่งทุก 2 วินาที

ลำดับเปิด: Broker -> Relay -> Subscriber -> Sensor

ลำดับปิด: Sensor -> Subscriber -> Relay -> Broker โดยกด `Ctrl+C` ในแต่ละ terminal

ค่าที่ทุกไฟล์ต้องตรงกัน

ค่า	Relay	Sender	เหตุผล
<code>relay_topic</code>	reliomq/messages	reliomq/messages	ไม่ตรงกัน Relay จะไม่ได้รับ envelope
<code>delivery_ack_topic</code>	reliomq/acks	reliomq/acks	ไม่ตรงกัน Sender จะไม่ได้รับ DeliveryAck
broker ทดลอง	localhost:1883	localhost:1883	ทั้งสองโปรแกรมต้องมองเห็น broker ตัวเดียวกัน

สร้าง Relay

เปิดโปรแกรมแก้ไขข้อความ เช่น VS Code หรือ Notepad สร้างไฟล์ชื่อ `relay.py` ในโฟลเดอร์ `reliomq-demo` แล้ววางโค้ดนี้:

```
import threading
from reliomq import Relay, RelayConfig

config = RelayConfig(
    source_host="localhost",
    destination_host="localhost",
    source_port=1883,
    destination_port=1883,
    source_client_id="demo-relay-source",
    destination_client_id="demo-relay-destination",
    relay_topic="reliomq/messages",
    delivery_ack_topic="reliomq/acks",
    log_level="INFO",
)

relay = Relay(config)
try:
    relay.connect()
    threading.Event().wait()
except KeyboardInterrupt:
    pass
finally:
    relay.disconnect()
```

โค้ดนี้ทำอะไร

ส่วน	อธิบายแบบง่าย
<code>source_host</code> <code>destination_host</code>	broker ที่รับจาก Sender และ broker ที่รับข้อมูลจริง ในการทดลองทั้งคู่คือ localhost
<code>relay_topic</code> <code>delivery_ack_topic</code>	ช่องรับ envelope และช่องส่งคำยืนยันกลับ ต้องตรงกับ Sender
<code>relay.connect()</code>	เชื่อม broker ทั้งสองฝั่ง แล้ว <code>threading.Event().wait()</code> ทำให้ Relay เปิดค้างไว้
<code>finally</code>	เมื่อกด Ctrl+C จะเรียก <code>relay.disconnect()</code> เพื่อปิดอย่างเรียบร้อย

client ID สองค่าต้องคนละชื่อ หากซ้ำกัน broker อาจตัดการเชื่อมต่ออีกฝั่งสลับไปมา

สร้าง Sensor Sender

สร้างไฟล์ `sensor_demo.py` ในโฟลเดอร์เดียวกัน แล้ววางโค้ดนี้:

```
import random
import time
from reliomq import Sender, SenderConfig

config = SenderConfig(
    host="localhost",
    port=1883,
    client_id="demo-sensor-01",
    outbox_path="./outbox/sensor01",
    relay_topic="reliomq/messages",
    delivery_ack_topic="reliomq/acks",
    log_level="INFO",
)

with Sender(config) as sender:    # default = FastMode()
    try:
        # LOOP: ส่งข้อมูลชุดใหม่ทุก 2 วินาทีจนกว่าจะกด Ctrl+C
        while True:
            data = {
                "temperature": round(random.uniform(24, 35), 2),
                "humidity": round(random.uniform(45, 75), 2),
            }
            message_id = sender.publish(
                "factory/demo/sensor01",
                data,
            )
            print("accepted:", message_id, data)
            print("pending:", sender.pending_count())
            time.sleep(2)
    except KeyboardInterrupt:
        print("sensor stopped")
```

อ่านโค้ดจากบนลงล่าง

บรรทัดสำคัญ	อธิบายแบบง่าย
<code>SenderConfig(...)</code>	บอก Sender ว่า broker อยู่ไหน ใช้ Outbox ที่ใด และคุยกับ Relay ผ่าน topic อะไร
<code>with Sender(config)</code>	เชื่อมต่อและเปิด background worker เมื่อออกจากบล็อกจะปิดให้อย่างเรียบร้อย
<code>while True</code>	นี่คือ loop ส่งข้อมูล: สร้างค่าใหม่ ส่ง แล้ววนกลับไปทำซ้ำจนกด Ctrl+C
<code>data = {...}</code>	สร้างข้อมูล Sensor จำลอง เป็น Python dictionary
<code>sender.publish(...)</code>	รับข้อความเข้า workflow และคืนหมายเลข <code>message_id</code>
<code>pending_count()</code>	จำนวนข้อความที่ Sender ยังรอให้ workflow จบ
<code>time.sleep(2)</code>	กำหนดความถี่ของ loop ให้รอ 2 วินาทีก่อนส่งชุดถัดไป

accepted ไม่ได้แปลว่าปลายทางได้รับแล้ว หมายถึง Sender รับข้อความเข้า workflow แล้วเท่านั้น เมื่อ DeliveryAck กลับมา ข้อความจึงจบงานและ pending จะลดลง

ลงมือรันทีละขั้น

1 เริ่ม broker

```
mosquitto -v
```

ถ้า broker ทำงานเป็น service อยู่แล้ว ข้ามขั้นนี้ได้

2 เริ่ม Relay ใน terminal ใหม่

```
cd reliomq-demo
venv\Scripts\Activate.ps1
python relay.py
```

บน Linux ใช้ `source venv/bin/activate` แทนบรรทัด activate ของ Windows

3 เปิดตัวดูปลายทาง

```
mosquitto_sub -h localhost -p 1883 -t "factory/demo/sensor01" -v
```

4 เริ่ม Sensor ใน terminal ใหม่

```
cd reliomq-demo
venv\Scripts\Activate.ps1
python sensor_demo.py
```

ผลที่ควรเห็น

หน้าต่าง	สิ่งที่ควรเกิด	แปลว่า
Sensor	พิมพ์ accepted: ... ทุก 2 วินาที	Sender รับข้อมูลเข้า workflow
Relay	มี log การรับและ forward	Relay เชื่อม broker และกำลังส่งต่อ
Subscriber	เห็น factory/demo/sensor01 และค่า JSON	ข้อมูลถึง application topic ปลายทาง
Sensor	pending ลดลงหลังช่วงสั้น ๆ	DeliveryAck กลับมาและข้อความจบงาน

สำเร็จครบ workflow เมื่อ subscriber เห็นข้อมูล และ Sender ได้ DeliveryAck จน pending ลด ไม่ใช่ดูเพียงคำว่า accepted

หากยังไม่ทำงาน ให้ไล่ตามจุดนี้

อาการ	ความหมายที่เป็นไปได้	ตรวจอย่างไร
Sensor connect ไม่ได้	เข้า source broker ไม่ได้	broker เปิดหรือยัง, host/port ถูกหรือไม่, firewall บล็อกหรือไม่
Sensor accepted แต่ Relay เจียบ	Relay ไม่ได้รับ envelope	Relay เปิดอยู่หรือไม่ และ relay_topic ตรงกันหรือไม่
Relay มี log แต่ subscriber เจียบ	ปลายทางหรือ application topic ไม่ตรง	ตรวจ destination_host และ topic ใน sender.publish()
Subscriber เห็นข้อมูล แต่ pending ไม่ลด	DeliveryAck กลับไม่ถึง Sender	ตรวจ delivery_ack_topic ทั้งสองไฟล์และ source broker connection
client ต่อแล้วหลุดสลับกัน	client ID ซ้ำ	ตั้ง ID ของ Sender และ Relay ทุก connection ให้ไม่ซ้ำ
Outbox error	เขียน disk ไม่ได้หรือ path ถูกใช้ร่วมกัน	ตรวจพื้นที่ว่าง/สิทธิ์ และแยก outbox_path ต่อ Sender process

เมื่อเข้าใจ flow แล้ว ค่อยเปลี่ยนค่าเหล่านี้

- localhost -> IP/hostname ของ broker จริง
- factory/demo/sensor01 -> application topic ของระบบคุณ
- client_id -> ชื่ออุปกรณ์ที่ไม่ซ้ำ
- outbox_path -> path แยกต่ออุปกรณ์และมีพื้นที่เพียงพอ
- เพิ่ม TLS, username/password และ ACL ตามนโยบายของ broker

เลือก Mode หลังจากส่งสำเร็จก่อน

Mode	คำอธิบายหนึ่งประโยค
FastMode ค่าเริ่มต้น	เริ่มใน RAM เร็วและเขียน disk น้อย แต่ข้อมูลล่าสุดอาจหายเมื่อไฟดับทันที
GroupMode	เขียนทุกข้อความแล้ว fsync เป็นชุด สมดุลระหว่างความเร็วและความทนทาน
DurableMode	fsync ต่อข้อความ เหมาะกับ alarm/event สำคัญ แต่ใช้ disk I/O มากที่สุด

สูตรจำ: เปิด Broker -> Relay -> ปลายทาง -> Sender; เมื่อข้อมูลถึง application topic แล้ว Relay ส่ง DeliveryAck กลับและ pending ลดลง
ดูการปรับโหมดใน reliomq Delivery Modes Guide